

Sistemet e Bazës së të Dhënave

SQL With, Views dhe Stored Procedures

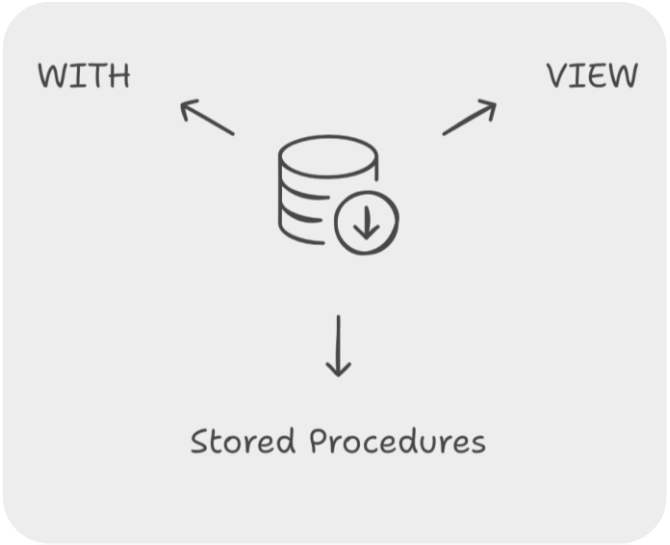
Elton Boshnjaku

SQL WITH, VIEW & Stored Procedures

WITH, **VIEW** dhe **Stored Procedures** janë veçori të avancuara të SQL-së që ofrojnë **optimizim**, **mirëmbajtje të lehtë** dhe **siguri të shtuar** për bazat e të dhënave, duke e bërë **menaxhimin e tyre më efikas dhe të qëndrueshëm**.

Pse është e rëndësishme zotërimi i këtyre veçorive?

- **Optimizimi i performancës**
- **Mirëmbajtja më e lehtë**
- **Siguria**



Çfarë është SQL WITH?

SQL WITH Clause, e njohur si **Common Table Expressions (CTEs)**, është një **mekanizëm që krijon një grup të përkohshëm rezultatesh** që **mund të përdoren brenda një query më të madhe**.

Përdorimi Kryesor:

- **Set i Përkohshëm** - Me WITH Clause, krijoni një tabelë të përkohshme vetëm për kohëzgjatjen e ekzekutimit të një query.
- **Ripërdorim në Query** - Kjo ju lejon të ripërdorni rezultatet e përkohshme në pjesë të ndryshme të një query.

```
1  WITH good_content AS (
2    SELECT
3      learn_sql
4    FROM dive.team
5  )
6
7  SELECT
8    learn_sql
9  FROM good_content
```

Sintaksa e SQL WITH

Sintaksa bazë e WITH është si më poshtë:

```
WITH cte_name (kolona1, kolona2) AS (
    SELECT kolona1, kolona2
    FROM emri_tabelës
    WHERE kusht
)
SELECT * FROM cte_name;
```

Shembull Praktik:

```
WITH StudentetElite (Emri, Nota) AS (
    SELECT Emri, Nota
    FROM Studentet
    WHERE Nota > 9
)
SELECT *
FROM StudentetElite;
```

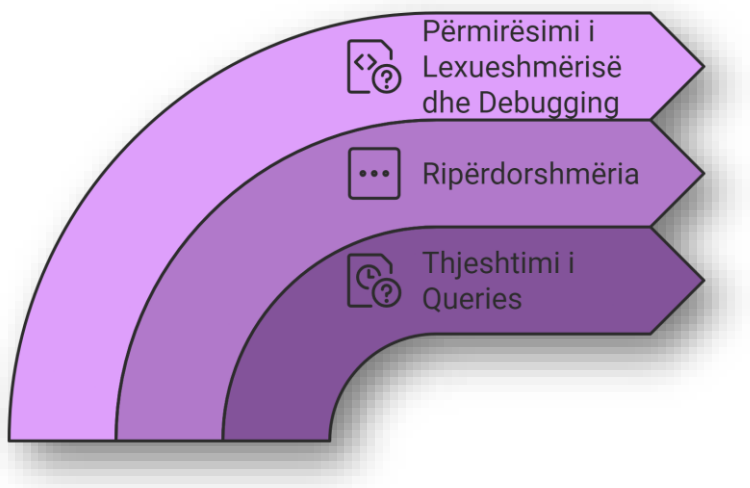
- Krijon një grup të përkohshëm rezultatesh të quajtur StudentetElite, që përmban vetëm studentët me nota më të mëdha se 9.
- Më pas, query kryesor tërheq të dhënat nga ky grup dhe i shfaq.

Përparësitë e SQL WITH

Thjeshton Nested Queries - WITH Clause ndan një query komplekse në pjesë logjike, duke thjeshtuar strukturën dhe duke e bërë kodin më të organizuar.

Ripërdorshmëria (Reusability) - Një Common Table Expression (CTE) mund të përdoret disa herë brenda një query, pa pasur nevojë për të shkruar përsëri të njëjtin kod.

Përmirëson Debugging dhe Lexueshmërinë - Pasi çdo WITH Clause është një njësi e ndarë, është më e lehtë të testoni dhe identifikoni gabimet.

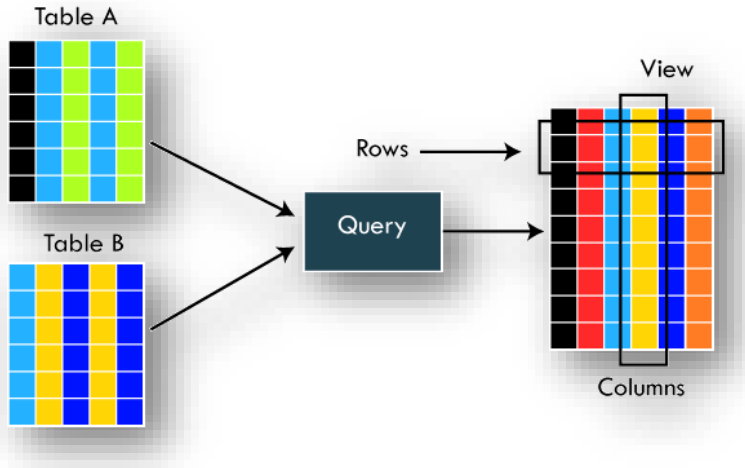


Çfarë janë SQL Views?

Një **View** është një **tabelë virtuale** **e krijuar nga një query SQL**.

Nuk ruan të dhëna fizikisht, por **mbështetet në tabelat bazë** nga të cilat nxirren të dhënat.

Ofron një **mënyrë të thjeshtë për të prezantuar** dhe **menaxhuar** të **dhënat**.



Përdorimi:

- Përdoret **për të shfaqur vetëm të dhënat e nevojshme**, duke fshehur detajet e tabelave bazë.
- **Kufizon aksesin në kolonat ose rreshtat e ndjeshëm** pa lejuar qasje të plotë në tabelat bazë.
- **Logjika komplekse mund të ruhet si një View dhe të ripërdoret** pa pasur nevojë për rishkrim.

Sintaksa e SQL VIEW

Për të krijuar një View, përdorni sintaksën e mëposhtme:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Shembull Praktik:

```
CREATE VIEW StudentetElite AS
SELECT Emri, Nota
FROM Studentet
WHERE Nota > 9;
```

Krijon një View të quajtur StudentetElite që përmban emrat dhe notat e studentëve me nota më të mëdha se 9.

Modifikimi i një View

Nëse keni nevojë të përditësoni një **View**, përdorni **ALTER VIEW**:

```
ALTER VIEW view_name AS
SELECT column1, column3
FROM table_name
WHERE condition;
```

Për të shtuar një kolonë ose **ndryshuar filtrimin në StudentetElite**, modifikoni logjikën e query-it me **ALTER VIEW**.

Fshirja e një View

Për të fshirë një View, përdorni komandën **DROP VIEW**:

```
DROP VIEW view_name;
```

Shembull:

```
DROP VIEW StudentetElite;
```

Ky komandë do të fshijë View-in StudentetElite nga baza e të dhënave.

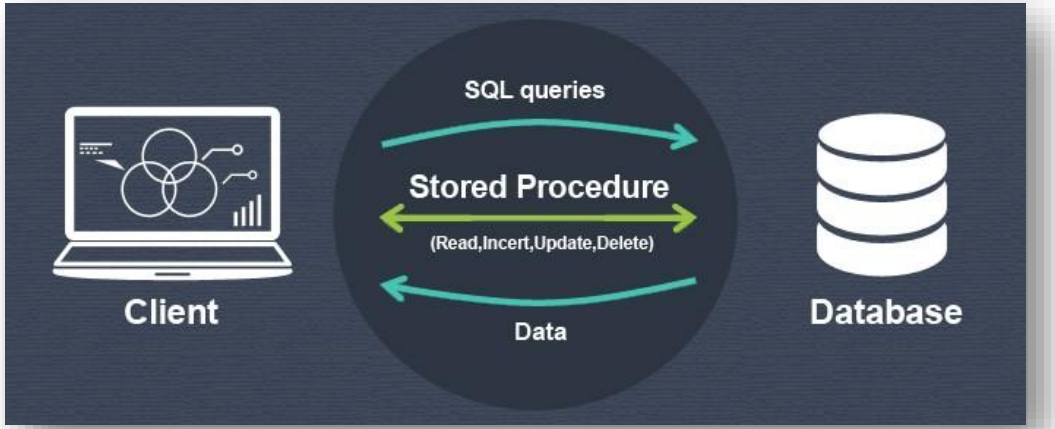
Stored Procedures

Stored Procedures në SQL janë një **koleksion i parapërpiluar i një ose më shumë deklaratave SQL që ruhen me një emër dhe ekzekutohen si njësi**.

Ato ruhen në bazën e të dhënave **dhe mund të ripërdoren shumë herë**, **duke përmirësuar efikasitetin**, **ripërdorshmërinë** dhe **sigurinë**.

Karakteristikat Kryesore:

- **Ekzekutim i Parapërpiluar**
- **Mbështetje për Parametra**
- **Logjikë Procedurale**



Sintaksa e Stored Procedures

Sintaksa Bazë:

```
CREATE PROCEDURE ProcedureName
AS
BEGIN
    -- Logjika ose komanda SQL këtu
END;
```

Shembull:

```
CREATE PROCEDURE GetAllProducts
AS
BEGIN
    SELECT ProductName, Price
    FROM Products;
END;
```

Ekzekutimi i Stored Procedure:

```
EXEC GetAllProducts;
```

- **BEGIN** dhe **END** përfshijnë logjikën procedurale të një Stored Procedure, duke siguruar që disa deklarata SQL trajtohen si një njësi e vetme.
- **EXEC** (ose **EXECUTE**) përdoret për të ekzekutuar një Stored Procedure në SQL Server.

Parametrat në Stored Procedures

Stored Procedures mbështesin **parametra Input dhe Output**, duke lejuar **ekzekutime dinamike bazuar në vlera të ndryshme të dhëna nga përdoruesi**.

Kjo i bën ato të përshtatshme për **logjikë fleksibile** dhe **operacione të personalizuara** mbi bazën e të dhënave.

Llojet e Parametrave në Stored Procedures:

- **Input Parameters (Parametra Hyrës):**
  - Përdoren **për të dërguar të dhëna** në Stored Procedure.
  - Sintaksa: **@parameter_name DataType**.

```
CREATE PROCEDURE GetEmployeeDetails (@EmployeeID INT)
AS
BEGIN
    SELECT Name, Position
    FROM Employees
    WHERE EmployeeID = @EmployeeID;
END;
```

- **Output Parameters (Parametra Dalës):**
  - Përdoren **për të kthyer vlera** nga Stored Procedure.
  - Sintaksa: **@parameter_name DataType OUTPUT**.

```
CREATE PROCEDURE GetTotalSales (@Year INT, @TotalSales DECIMAL(10, 2) OUTPUT)
AS
BEGIN
    SELECT @TotalSales = SUM(SalesAmount)
    FROM Sales
    WHERE YEAR(SalesDate) = @Year;
END;
```

Sintaksa për Stored Procedures me Parametra

```
CREATE PROCEDURE ProcedureName (@param1 DataType, @param2 DataType OUTPUT)
AS
BEGIN
    -- Logjika SQL që përdor parametrat
END;
```

Ekzekutimi i Stored Procedures me Parametra:

```
EXEC ProcedureName @param1 = 'Value1';
```

```
DECLARE @OutputValue DECIMAL(10, 2);
EXEC ProcedureName @param1 = 'Value1', @param2 = @OutputValue OUTPUT;
SELECT @OutputValue; -- Shfaq vlerën e kthyer
```

Ndryshimi i Stored Procedures

Stored Procedures mund të modifikohen për të përditësuar **logjikën**, **parametrat**, ose **strukturën** e tyre pa krijuar një të re nga e para.

SQL Server ofron komandën **ALTER PROCEDURE** për të ndryshuar procedurat ekzistuese.

Sintaksa për **Ndryshimin e Stored Procedures**:

```
ALTER PROCEDURE ProcedureName
AS
BEGIN
    -- Logjika e re ose e përditësuar
END;
```

Shembull:

```
CREATE PROCEDURE GetEmployeeDetails (@EmployeeID INT)
AS
BEGIN
    SELECT EmployeeName, Position
    FROM Employees
    WHERE EmployeeID = @EmployeeID;
END;
```

```
ALTER PROCEDURE GetEmployeeDetails (@EmployeeID INT)
AS
BEGIN
    SELECT EmployeeName, Position, Salary
    FROM Employees
    WHERE EmployeeID = @EmployeeID;
END;
```

Fshirja e Stored Procedures

Një **Stored Procedure mund të fshihet** nga baza e të dhënave duke përdorur komandën **DROP**. Ky veprim heq përfundimisht procedurën dhe të gjitha lidhjet e saj me bazën e të dhënave. Përdoret kur një Stored Procedure nuk është më e nevojshme ose ka një version të ri që e zëvendëson.

Sintaksa për **Fshirjen e Stored Procedures**:

```
DROP PROCEDURE ProcedureName;
```

Shembull:

```
CREATE PROCEDURE GetEmployeeDetails (@EmployeeID INT)
AS
BEGIN
    SELECT EmployeeName, Position
    FROM Employees
    WHERE EmployeeID = @EmployeeID;
END;
```

```
DROP PROCEDURE GetEmployeeDetails;
```

Logjika Procedurale në Stored Procedures

Stored Procedures mbështesin **elemente të logjikës procedurale** si:

- **Deklarimi** dhe **përdorimi i variablave**.
- **Përdorimi** i **IF...ELSE** për të marrë vendime **bazuar në kushte**.
- **Strukturat** e tjera **kontrolluese** si **LOOPS** (p.sh., **WHILE**).

```sql
CREATE PROCEDURE CalculateDiscount
    @OrderID INT
AS
BEGIN
    DECLARE @OrderTotal DECIMAL(10, 2);
    DECLARE @Discount DECIMAL(10, 2);

    -- Merr totalin e porosisë
    SELECT @OrderTotal = TotalAmount FROM Orders WHERE OrderID = @OrderID;

    -- Vendos zbritjen bazuar në totalin
    IF @OrderTotal >= 1000
        SET @Discount = @OrderTotal * 0.10;
    ELSE
        SET @Discount = @OrderTotal * 0.05;

    PRINT 'Zbritja është: ' + CAST(@Discount AS NVARCHAR);
END;

-- Ekzekutimi
EXEC CalculateDiscount @OrderID = 202;
```

Logjika Procedurale në Stored Procedures

Deklarimi i Variablave:

Një **variabël** në SQL Server **është një objekt i përkohshëm** që ruan një vlerë të vetme dhe **përdoret për të kryer llogaritje ose për të ruajtur rezultate të përkohshme gjatë ekzekutimit të një Stored Procedure** apo **query**.

Variablat deklarohen duke përdorur fjalën kyçe **DECLARE**, ndersa ju vendoset vlerë duke përdorur fjalën kyçe **SET**.

Sintaksa:

```
DECLARE @VariableName DataType;
SET @VariableName = Value;
```

```
DECLARE @TotalSales DECIMAL(10, 2);
SET @TotalSales = 0.00;

SELECT @TotalSales = SUM(SalesAmount) FROM Sales WHERE Year(SalesDate) = 2024;
PRINT 'Totali i Shitjeve për 2024 është: ' + CAST(@TotalSales AS NVARCHAR);
```

Logjika Procedurale në Stored Procedures

IF… Else:

Strukturat **IF…ELSE** përdoren për të **marrë vendime bazuar në një kusht të caktuar**.

ELSE IF shton **kushte të tjera për t'u testuar nëse kushti fillestar është i pavërtetë**.

Sintaksa:

```
IF Condition
    BEGIN
        -- Blloku i kodit kur kushti është TRUE
    END
ELSE IF AnotherCondition
    BEGIN
        -- Blloku i kodit për kushtin tjetër
    END
ELSE
    BEGIN
        -- Blloku i kodit kur asnjë kusht nuk plotësohet
    END
```

```
DECLARE @Salary DECIMAL(10, 2);
SET @Salary = 60000;

IF @Salary > 80000
    PRINT 'Punonjësi ka pagë të lartë.';
ELSE IF @Salary BETWEEN 50000 AND 80000
    PRINT 'Punonjësi ka pagë mesatare.';
ELSE
    PRINT 'Punonjësi ka pagë të ulët.';
```

Logjika Procedurale në Stored Procedures

WHILE Loop:

WHILE përdoret për të krijuar **cikle që përsërisin një bllok të caktuar kodi sa herë që një kusht është i vërtetë**.

Sintaksa:

```
WHILE Condition
    BEGIN
        -- Blloku i kodit që ekzekutohet gjatë përsëritjes
    END
```

```
DECLARE @Counter INT;
SET @Counter = 1;

WHILE @Counter <= 5
    BEGIN
        PRINT 'Kjo është përsëritja numër: ' + CAST(@Counter AS NVARCHAR);
        SET @Counter = @Counter + 1;
    END
```



Do you have any questions?

elton.boshnjaku@ubt-uni.net
www.ubt-uni.net

Referencat

Fundamentals of Database Systems
Authors: Ramez Elmasri and Shamkant B. Navathe
Edition: 7th Edition
Year: 2016

Practical SQL: A Beginner's Guide to Storytelling with Data
Author: Anthony DeBarros
Edition: 1st Edition
Year: 2018

SQL All-in-One For Dummies
Author: Allen G. Taylor
Edition: 3rd Edition
Year: 2019